Chained Elo Can Point the Wrong Way: Evaluating a Laptop-Scale AlphaZero for 9×9 Go

Anonymous submission

Abstract

Small self-play runs, of the kind used in teaching, hobby engines and research prototypes, cannot afford large evaluations, so progress is often tracked by matching each new network against its predecessor and summing the Elo differences ("chained Elo"). We train an AlphaZero-style 9×9 Go engine from scratch on one 8-core laptop CPU (44 generations, 7,040 self-play games) and report a case where this signal pointed the wrong way: chained Elo fell after generation 25 and ranked generation 34 below generations 19 and 25, yet in direct matches generation 34 beat them in 79/80 and 66/80 games. Anchored matches against fixed earlier and later generations (2,080 games) show steady improvement throughout. Match noise contributes, but cannot explain the size of the error. Replaying the same consecutive matches with more search per move recovers the correct direction and agrees with the anchored games. In this run, chained Elo misled mainly because the per-generation matches were too cheap. We describe the laptop-scale training pipeline, whose rules and search we verified to produce identical results in Python, Swift and the browser. We also report an external calibration: the final network won 0/40 games against GNU Go at its weakest level. Finally, we recommend a cheap anchored protocol for evaluating small runs.

1 Introduction

AlphaGo Zero and AlphaZero learn to play board games from nothing but the rules, by alternating self-play with tree search and training a policy/value network on the resulting games (Silver et al. 2017, 2018). The original systems used thousands of accelerators, but the algorithm is simple enough that small versions are widely built: in courses, in hobby engines, and as research prototypes before a large run. At that scale the budget for *evaluation* is tiny, and the most common progress signal is also the cheapest one: after each generation, play a short match between the new network and the previous one, convert the score to an Elo difference (Elo 1978), and add it to a running total. We call this total *chained Elo*.

This paper reports a run where chained Elo told the wrong story. We trained a 9×9 Go engine for 44 generations on one laptop CPU. Chained Elo peaked at generation 25 (+418) and then drifted down, putting generation 34 (+74) below both generation 25 (+418) and generation 19 (+127). Taken at face value, training had stalled or regressed. In direct 80-game matches, however, generation 34 beat generation 25 in 66/80 games and generation 19 in 79/80, and the final generation 44 beat generation 34 in 59/80.

Contributions.

- An evaluation of a complete small run against fixed anchors (2,080 games), showing steady improvement where chained Elo shows a decline (Figure 3).
- An analysis of why chained Elo failed. Match noise grows with the number of generations, but the observed drop lies -3.9 standard deviations from what noise around the true gain would produce. Replaying the same consecutive pairs at the anchored settings (64 rather than 32 simulations) gives gains that agree with the anchored matches (Figure 4). From this we derive a cheap protocol for small runs (Section 6).
- An open AlphaZero pipeline that trains on one laptop CPU. Its rules and search were checked to give identical results in Python, Swift with Core ML, and the browser with ONNX Runtime, so the same engine ships as a native app and a web page (Table 2).
- An external calibration against GNU Go, which shows how far such a network remains from a classical engine (Section 5).

2 Related Work

AlphaZero and its reimplementations. AlphaGo combined deep networks with Monte Carlo tree search (Silver et al. 2016); AlphaGo Zero and AlphaZero removed human data and hand-crafted features (Silver et al. 2017, 2018). Their search descends from UCT and its prior-guided variant PUCT (Kocsis and Szepesvári 2006; Rosin 2011). Open reimplementations reproduce the result at large scale (Tian et al. 2019), make self-play far more sample-efficient (Wu 2019), or provide AlphaZero inside general game frameworks (Lanctot et al. 2019). We borrow KataGo's playout-cap randomization (Wu 2019). We work at the opposite end of the scale: one CPU, and a deliberately small network.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

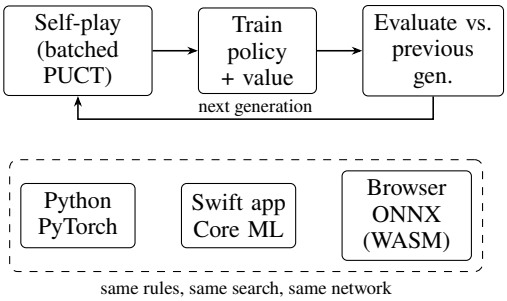


Figure 1: One training loop on a laptop CPU; the same rules, search and trained network run in three runtimes. Chained Elo comes from the "evaluate vs. previous generation" step.

Rating and evaluating agents. Elo ratings and the Bradley–Terry model turn pairwise results into a scale (Elo 1978; Bradley and Terry 1952). Whole-history rating fits ratings that change over time (Coulom 2008). Evaluation by pairwise results can mislead when games are not transitive (Balduzzi et al. 2018; Czarnecki et al. 2020), and point estimates from a few reinforcement-learning runs can be unreliable without interval estimates (Agarwal et al. 2021). AlphaGo Zero tracked progress with large evaluation matches against earlier players (Silver et al. 2017). Recent limited-compute systems rate their engines against fixed external references: a ladder of a classical engine at fixed node counts (Braun 2026), or a round robin anchored to a solver (Sherwood, Aybar, and Kaplan 2026); voting-based aggregation is a further alternative to Elo (Lanctot et al. 2023). These anchors need an engine strong enough to score against them. Ours lost every game to GNU Go at its weakest level (Section 5), so an external ladder could not have separated its generations. We study the cheap internal signal many small runs rely on instead, consecutive-generation matches, and show how and why it can mislead.

3 A Laptop-Scale AlphaZero

Game and representation. We play 9×9 Go with simple ko, no suicide and Tromp–Taylor area scoring (komi 7.5); games end after two passes or 162 moves. The network sees six planes: each side's stones now and one move ago, the side to move, and a constant plane.

Network and search. A residual network (He et al. 2016) with 188,173 parameters (Table 1) outputs move logits for the 81 points and pass, and a value in $[-1, 1]$. Search is PUCT (Rosin 2011) as in AlphaZero, with one change from Leela Zero: an unvisited move is valued at the parent's value minus a reduction that grows with the prior probability already explored (first-play urgency).

Self-play and training. Ten worker processes each play 16 games in lock step. Every simulation step evaluates one leaf per game in a single network batch, which keeps a CPU busy without virtual loss or threads. Following KataGo (Wu 2019), a quarter of moves receive a full search and become training samples; the rest get a quarter of the simulations and only advance the game. The network trains with AdamW on a sliding window of recent positions, with all eight board symmetries, on cross-entropy to the search's visit distribution plus squared error to the game result. Each generation then plays the previous one in a 40-game match at 32 simulations a move; the summed Elo differences form the chained Elo of the training log.

Setting	Value
Board, komi, move cap	9×9, 7.5, 162
Input planes	6
Residual blocks × channels	4 × 48
c_{puct}, FPU reduction	1.5, 0.25
Dirichlet α, ε	0.12, 0.25
Sims: full / fast search	160 / 40
Full-search probability	0.25
Temperature-1 moves	10
Games / generation, workers	160, 10
Batch, learning rate (AdamW)	256, 0.001
Replay window (positions)	250,000
Chained eval: games, sims	40, 32

Table 1: Settings of the run, read from the training code and its saved configuration.

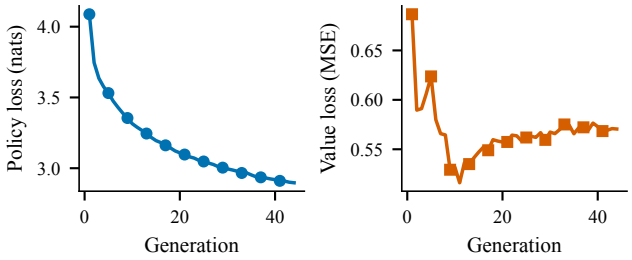


Figure 2: Training losses fall steadily across generations: policy cross-entropy from 4.09 to 2.90 nats, and value MSE from 0.69 to 0.57. One run.

Hardware and budget. Everything ran on one Intel Core i9-9980HK (8 cores, 16 threads, no GPU). The run took 44 generations of 160 games each (7,040 games, about 11.0 hours of wall-clock time). That time is approximate: other software shared the machine, and one generation's self-play took anywhere from 197 to 2,903 seconds. Figure 2 shows the losses.

4 Evaluating a Small Self-Play Run

Chained Elo and its noise. Let p_g be the score of generation g against $g-1$ in an n-game match. Chained Elo is $E_g = \sum_{i \le g} 400 \log_{10}\big(p_i/(1-p_i)\big)$, with p_i clipped to $[0.02, 0.98]$. Each term has a standard error of about $\frac{400}{\ln 10} \cdot \frac{\sqrt{0.25/n}}{0.25}$ at $p = 0.5$, which is 55 Elo for $n = 40$. The errors are independent, so they add like a random walk: after 44 generations the spread from noise alone is ±364 Elo (one standard deviation). Two generations far apart can therefore be ranked in the wrong order even when every step

is unbiased. Section 5 shows that noise alone does not explain our run.

Anchored matches. To measure progress without chaining, we play every fourth generation $(0, 4, \ldots, 44)$ against the next two sampled generations and against the final generation 44. This gives 30 pairs of 40 games (1,200 games), each at 64 simulations a move, with colors balanced and moves sampled in proportion to visits for the first six moves. Together with eleven 80-game checks played between release candidates, that gives 2,080 games. We report the results in two ways:

- *model-free*: each sampled generation's score against the final one, with 95% Wilson intervals;
- a Bradley–Terry fit (Bradley and Terry 1952) on all games on the Elo scale (generation $0 = 0$), with a weak Gaussian prior (s.d. 1000) to keep one-sided results finite, and 95% bootstrap intervals from resampling games within each match.

Consecutive replay. To separate the effect of short matches from the effect of comparing adjacent generations, we replay the consecutive pairs from generation 24 onwards at the anchored settings (20 pairs, 40 games each, 64 simulations).

5 Results

Chained Elo misranks the run. Figure 3a shows the training log's chained Elo: a peak of +418 at generation 25, a decline to +74 at generation 34, and +252 at the end. Every anchored measurement disagrees.

- Head to head, generation 34 beats generation 25 in 66/80 games and generation 19 in 79/80, and generation 44 beats generation 34 in 59/80.
- Against the final generation, the scores of generations $0, 4, \ldots, 40$ are 0, 0, 1, 0, 0, 0, 2, 6, 6, 8, 16 wins out of 40: flat at zero while the earlier networks are far weaker, then rising steadily from generation 24 (Figure 3b), with no sign of the decline after generation 25.
- The Bradley–Terry ratings increase with generation apart from two less-constrained points (generations 10 and 19, which appear only in a few long-range matches) (Figure 3c).

The fit is approximate. Its Pearson χ^2 is 323 over 41 matches with 17 free ratings. Its residuals are systematic: in close pairs the newer network wins more often than predicted, and in very lopsided pairs the weaker network still wins a few games, mostly through the randomized opening. We therefore do not quote a total Elo gain from the fit.

Why: noise contributes, but the cheap matches are the main cause. Pairs four generations apart give the newer network a pooled win rate of 0.786, or about 57 Elo per generation over the whole run. Over generations 24–44 alone, the rate is 33 Elo per generation, a per-generation win rate of 0.547 (95% interval 0.529–0.565). At that rate, chained Elo should have risen by about +295 from generation 25 to 34, with a noise standard deviation of 3×55 Elo. Instead it fell by 344, -3.9 standard deviations from the expectation, so noise alone is an unlikely explanation.

Runtime	ms	sims/s	Same	Max $\lvert\Delta\rvert$
PyTorch	974	411	15/15	–
Core ML	**534**	**750**	15/15	5.5×10^{-6}
ONNX (web)	975	410	15/15	7.6×10^{-6}

Table 2: The same search on three runtimes: PyTorch (Python trainer), Core ML (Swift app) and ONNX Runtime Web (browser, WebAssembly). Columns: median time (ms) for a 400-simulation search over 15 positions (best of three runs each, one leaf per network call, one CPU thread); positions whose root visit counts equal PyTorch's exactly; largest network output difference from PyTorch.

The training log's matches were cheaper than our anchored ones: 32 simulations a move instead of 64. Replaying the consecutive pairs from generation 24 onwards at 64 simulations gives the newer network 0.537 of the games (95% interval 0.503–0.572), against 0.493 for the same pairs in the log. That is consistent with the 0.547 implied by pairs four apart over the same generations (Figure 4). Chained over generations 25 to 34, the replay gives +159 Elo, the correct direction (the head-to-head result implies +269), where the log gave -344. Over generations 24–44 it gives +551, where the log gave -114.

So comparing adjacent generations is not misleading in itself. In this run the per-generation matches were too shallow to separate networks that, at deployment-like search depth, differ by a few dozen Elo, and summing their noisy, compressed differences hid the trend. On any one pair the difference between the log and the replay is within noise, so we cannot fully separate the effect of search depth from the play of chance with one run. But every 64-simulation measurement in Figure 4 points the same way.

External calibration. Against GNU Go 3.8 at level 1 (its weakest), with Chinese rules, komi 7.5 and `--capture-all-dead` so that our area scoring is fair to it, generation 44 at 400 simulations a move won 0/40 games, with a median losing margin of 81.0 points. GNU Go's own scoring named the same winner in 40/40 games. The internal gains above are real, but in absolute terms the network remains far weaker than a classical engine at its weakest setting.

One engine, three runtimes. The rules are implemented three times: in Python for training, in Swift for the macOS app and in JavaScript for the browser. All three agree exactly on the board, ko point, legal moves, score and input planes for 433 positions taken from 40 random games, generated by the Python rules. The trained network, exported to Core ML and to ONNX, matches PyTorch to within 5.5×10^{-6} and 7.6×10^{-6} on random inputs. On 15 positions from a self-play game, searched from scratch for 400 simulations each, the root visit counts were identical in all positions on all three runtimes (Table 2), so the app and the web page make exactly the moves the trainer would.

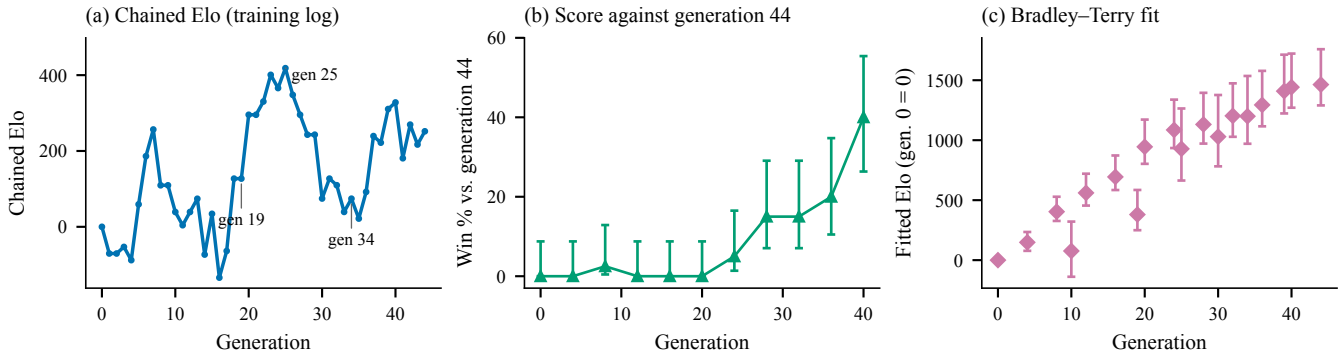


Figure 3: Chained Elo says training stalled after generation 25; anchored games say it kept improving. (a) Chained Elo from the training log (40-game matches against the previous generation, 32 simulations). (b) Score of every fourth generation against the final generation 44 (40 games each, 64 simulations), with 95% Wilson intervals. (c) Bradley–Terry ratings fitted to all 2,080 head-to-head games, with 95% bootstrap intervals. The model fits poorly at large rating gaps (see text), so read (c) as a ranking, not a scale.

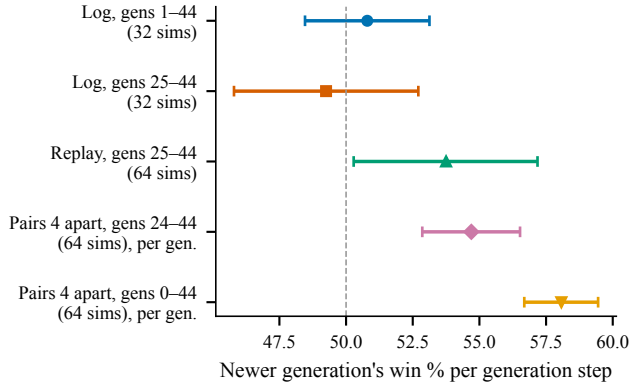


Figure 4: At 64 simulations, consecutive-generation matches agree with pairs four apart; the training log's 32-simulation matches measured almost no gain. Each row is the newer network's win rate for a single generation step, with 95% Wilson intervals; results for pairs four apart are converted to a single step through the Elo scale. Compare rows over the same generations: gains are larger early in training.

6 Lessons for Small Runs

Evaluate against anchors, not only the predecessor. A protocol that fits a small budget and would have caught our false decline:

1. Play evaluation games with at least the search depth you will deploy. Matches that are too shallow compress real differences toward a draw.
2. Keep the consecutive-generation match for gating or monitoring if useful, but do not sum it into a strength curve. Every few generations, play the new network against a fixed set of anchors instead (an early generation, a mid-run generation and the most recent release).
3. Report scores against the anchors with confidence intervals, and promote a network only when it beats the current release head to head.

In our run, a single row of Figure 3b, the score against one late anchor, already shows the trend that chained Elo hid. It cost 40 games per sampled generation.

Engineering notes. Lock-step batched search keeps a CPU-only machine busy without threads. Playout-cap randomization makes more games per hour (Wu 2019). Worker pools need timeouts. Python's `multiprocessing.Pool` replaces a worker that dies but drops that worker's job, which once stalled our run for two hours on an overloaded machine. Pool calls now time out and are retried on fresh workers. Running the rules in more than one language paid for itself through shared test positions generated by the trainer.

7 Limitations and Conclusion

Limitations. This is one training run with one seed, on 9×9 Go, with a small network. We do not know how often chained Elo misleads in other runs, games or network sizes; the analysis of noise is general, but the size of the effect of shallow evaluation may not be. We compared only 32 and 64 simulations a move, and replayed only generations 24–44. On a single pair, the log and the replay differ by less than the noise, so the attribution rests on the aggregate. Wall-clock times were measured on a machine shared with other work. We played no games against humans, and the GNU Go comparison used a single setting.

Conclusion. On a small AlphaZero run, summing Elo from short matches between consecutive generations suggested a stall and decline. Anchored head-to-head games showed steady improvement. The disagreement is too large to be match noise alone. Replaying the same consecutive matches with more search per move recovered the correct direction and agreed with the anchored games. For small runs we recommend evaluating at least at deployment search depth, and tracking scores against a few fixed anchors, which costs little. The training pipeline, the evaluation data and the identical-across-runtimes engine are released with the paper **TODO: add the anonymized artifact link for review; the public**

repository URL for camera-ready.

Use of AI tools. The implementation (training pipeline, rules engines, apps, evaluation scripts and benchmarks) was written with an AI coding assistant (Claude Code) under the authors' direction and checked by the tests and parity checks in Section 5. The same assistant, with a paper-writing skill, drafted text, wrote the scripts that generate figures, tables and in-text numbers from the logged data, and formatted and verified references. The authors reviewed all content and are responsible for it. **TODO: author: confirm this statement matches what you did, and edit as needed.**

References

Agarwal, R.; Schwarzer, M.; Castro, P. S.; Courville, A.; and Bellemare, M. G. 2021. Deep Reinforcement Learning at the Edge of the Statistical Precipice. In *Advances in Neural Information Processing Systems 34*.

Balduzzi, D.; Tuyls, K.; Pérolat, J.; and Graepel, T. 2018. Re-evaluating Evaluation. In *Advances in Neural Information Processing Systems 31*.

Bradley, R. A.; and Terry, M. E. 1952. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4): 324–345.

Braun, B. 2026. Engineering Efficient Self-Play Chess: Search, Replay, and Throughput Under Limited Compute. arXiv:2609.37447.

Coulom, R. 2008. Whole-History Rating: A Bayesian Rating System for Players of Time-Varying Strength. In *Computers and Games*, volume 5131 of *Lecture Notes in Computer Science*, 113–124. Springer.

Czarnecki, W. M.; Gidel, G.; Tracey, B.; Tuyls, K.; Omidshafiei, S.; Balduzzi, D.; and Jaderberg, M. 2020. Real World Games Look Like Spinning Tops. In *Advances in Neural Information Processing Systems 33*.

Elo, A. E. 1978. *The Rating of Chessplayers, Past and Present*. New York: Arco.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep Residual Learning for Image Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.

Kocsis, L.; and Szepesvári, C. 2006. Bandit Based Monte-Carlo Planning. In *Machine Learning: ECML 2006*, volume 4212 of *Lecture Notes in Computer Science*, 282–293. Springer.

Lanctot, M.; Larson, K.; Bachrach, Y.; Marris, L.; Li, Z.; Bhoopchand, A.; Anthony, T.; Tanner, B.; and Koop, A. 2023. Evaluating Agents using Social Choice Theory. arXiv:2312.03121.

Lanctot, M.; Lockhart, E.; Lespiau, J.-B.; Zambaldi, V.; Upadhyay, S.; Pérolat, J.; Srinivasan, S.; Timbers, F.; Tuyls, K.; Omidshafiei, S.; Hennes, D.; Morrill, D.; Muller, P.; Ewalds, T.; Faulkner, R.; Kramár, J.; De Vylder, B.; Saeta, B.; Bradbury, J.; Ding, D.; Borgeaud, S.; Lai, M.; Schrittwieser, J.; Anthony, T.; Hughes, E.; Danihelka, I.; and Ryan-Davis, J. 2019. OpenSpiel: A Framework for Reinforcement Learning in Games. arXiv:1908.09453.

Rosin, C. D. 2011. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3): 203–230.

Sherwood, J.; Aybar, B.; and Kaplan, B. 2026. Frontier Coding Agents Can Now Implement an AlphaZero Self-Play Machine Learning Pipeline For Connect Four That Performs Comparably to an External Solver. arXiv:2604.25067.

Silver, D.; Huang, A.; Maddison, C. J.; Guez, A.; Sifre, L.; van den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; Dieleman, S.; Grewe, D.; Nham, J.; Kalchbrenner, N.; Sutskever, I.; Lillicrap, T.; Leach, M.; Kavukcuoglu, K.; Graepel, T.; and Hassabis, D. 2016. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587): 484–489.

Silver, D.; Hubert, T.; Schrittwieser, J.; Antonoglou, I.; Lai, M.; Guez, A.; Lanctot, M.; Sifre, L.; Kumaran, D.; Graepel, T.; Lillicrap, T.; Simonyan, K.; and Hassabis, D. 2018. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419): 1140–1144.

Silver, D.; Schrittwieser, J.; Simonyan, K.; Antonoglou, I.; Huang, A.; Guez, A.; Hubert, T.; Baker, L.; Lai, M.; Bolton, A.; Chen, Y.; Lillicrap, T.; Hui, F.; Sifre, L.; van den Driessche, G.; Graepel, T.; and Hassabis, D. 2017. Mastering the game of Go without human knowledge. *Nature*, 550(7676): 354–359.

Tian, Y.; Ma, J.; Gong, Q.; Sengupta, S.; Chen, Z.; Pinkerton, J.; and Zitnick, L. 2019. ELF OpenGo: an analysis and open reimplementation of AlphaZero. In *Proceedings of the 36th International Conference on Machine Learning*.

Wu, D. J. 2019. Accelerating Self-Play Learning in Go. arXiv:1902.10565.